STRIDE: Learning Humanoid Locomotion in Minutes with Acceleration-Based Rewards

Nico Bohlinger[1] **Jan Peters**[1,2,3,4]
[1]Technical University of Darmstadt, Germany [2]Robotics Institute Germany (RIG)
[3]German Research Center for AI (DFKI) [4]hessian.AI

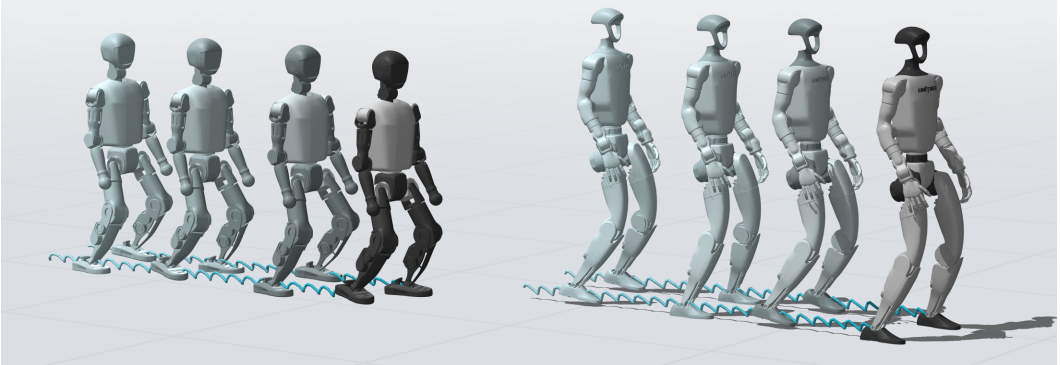


Figure 1: The Booster T1 (left) and the Unitree G1 (right) walk with policies that STRIDE learned in 14 minutes worth of collected samples.

Abstract: Learning locomotion directly on a humanoid robot is a promising direction to avoid the sim-to-real gap. However, locomotion rewards in Reinforcement Learning (RL) typically track the base velocity, which cannot be measured onboard without an external motion capture system or well-calibrated state estimator. This is why we introduce STRIDE, which learns humanoid locomotion from minutes of samples with a reward that only requires the onboard Inertial Measurement Unit (IMU). Instead of tracking the base velocity, STRIDE rewards the robot for following a commanded acceleration profile. To learn efficiently from this reward, we use the off-policy RL algorithm XQC and combine a high gain setting for the joint position controllers with left-right symmetry augmentation. In simulation, STRIDE learns to walk on the Unitree G1 and Booster T1 humanoids within 14 minutes of samples. Compared to learning without these two components, it reduces the falls during training by 81%, while without them the robot learns only in over an hour.

Keywords: Humanoid Locomotion, Reinforcement Learning, Efficient Learning

1 Introduction

Reinforcement Learning (RL) has become the standard approach for legged locomotion [1, 2, 3], where policies are trained in thousands of parallel environments in simulation and then transferred to the real robot [4, 5, 6, 7]. This transfer requires accurate models, extensive domain randomization and often an adaptation to the physical robot [8, 9]. Learning directly on the robot avoids this gap, and quadrupeds already learn to walk within minutes of real-world interaction [10, 11]. Yet, two properties of humanoids make learning on the robot especially difficult. First, the locomotion rewards typically used in prior work track the base velocity, which has to come from a motion capture system or from a well-calibrated state estimator [12]. But even such a state-estimator can

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

still introduce errors and delays, or systematically underestimate the velocity [11]. Second, bipeds are inherently unstable, so they need large amounts of samples to learn from and fall frequently. Every fall on a real robot requires a reset and wears down the hardware.

This is why we propose STRIDE, which learns humanoid locomotion from minutes of samples with a reward that only requires the onboard Inertial Measurement Unit (IMU). Instead of tracking the base velocity, STRIDE rewards the robot for following a commanded acceleration profile and for holding the velocity that it estimates by integrating its own acceleration. To learn efficiently from this reward, we use a high gain setting for the joint position controllers and train an off-policy RL algorithm on symmetry-augmented copies of the collected transitions. In simulation, STRIDE learns to walk on the Unitree G1 and Booster T1 humanoids within 14 minutes worth of samples. Compared to learning with a low gain setting and without symmetries, it reduces the falls during training by 81%, while only half of the runs without these two components learn to walk within 83 minutes at all.

2 Related Work

Locomotion has been learned directly on quadrupeds with maximum entropy RL [13] and with world models [14, 15], and more sample-efficient off-policy algorithms reduced the training time to minutes [10, 11]. This efficiency builds on better-conditioned critic learning [16, 17] and higher Update-To-Data (UTD) ratios [18], while the action space and the low-level controller also influence how fast a policy learns [19]. Structural knowledge reduces the number of required samples further, such as the left-right symmetry of the robot [20] or the periodic gait prior of a Central Pattern Generator (CPG) [21]. STRIDE combines these ideas for humanoids, which are considerably less stable than quadrupeds, and removes the need for a base velocity estimate. Finally, embodiment-aware architectures scale a single policy to many different robots in simulation [6, 7, 22, 23], and we take a first step toward learning such a shared policy from the limited data of a few robots.

3 Learning Humanoid Locomotion Efficiently

We propose to learn humanoid locomotion with a reward that is computed only from the IMU and to make learning from this reward efficient enough for the real world. We formulate the task as a Markov Decision Process, in which the policy $\pi_\theta(a_t \mid o_t)$ maps the observation o_t to position targets a_t for the leg joints at every control step t. The observation consists of the joint positions and velocities, the previous action, and the linear acceleration, the angular velocity and the gravity direction from the IMU. It also contains the commanded velocity, the reference velocity and the goal acceleration, which we introduce in Section 3.1 and which let the policy see what it is rewarded for. Neither the observation nor the reward requires a measured base velocity.

3.1 Acceleration-Based Reward

An IMU does not measure the velocity of the robot but the linear acceleration α_t of its base, so we formulate the locomotion task in terms of accelerations in the horizontal plane of the IMU frame. To turn a commanded velocity v^{c} into an acceleration profile that the robot can follow, we define a reference velocity v_t^{r} that approaches the commanded velocity with a bounded acceleration. At the same time, the robot keeps an onboard velocity estimate $\hat{v}_t$ by integrating its acceleration as

$$v_t^{\mathrm{r}} = v_{t-1}^{\mathrm{r}} + \mathrm{clip}_{\alpha_{\max}\Delta t}\big(v^{\mathrm{c}} - v_{t-1}^{\mathrm{r}}\big) \qquad \text{and} \qquad \hat{v}_t = \hat{v}_{t-1} + \alpha_t\,\Delta t, \tag{1}$$

where Δt is the control period and $\alpha_{\max}$ is the maximum acceleration. A Proportional-Integral (PI) law then computes a goal acceleration g_t that drives the estimate toward the reference as

$$g_t = \mathrm{clip}_{\alpha_{\max}}\Big(k_{\mathrm{P}}\,e_t + k_{\mathrm{I}}\sum\nolimits_{i\le t} e_i\,\Delta t\Big) \qquad \text{with} \qquad e_t = v_t^{\mathrm{r}} - \hat{v}_t, \tag{2}$$

where k_{P} and k_{I} are the gains. The integral term keeps increasing the goal acceleration as long as there is a velocity error, so the robot does not stay below the commanded velocity. Walking

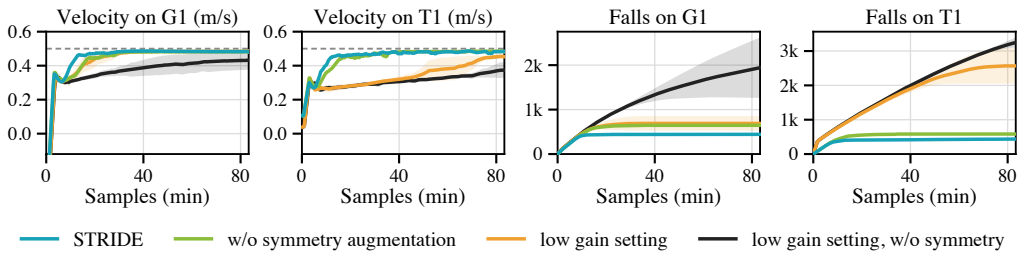


Figure 2: Forward velocity and number of falls during training for STRIDE and its ablations, shown as mean and standard deviation over all runs. The dashed line shows the commanded velocity.

accelerates and decelerates the base within every step the robot takes, and a reward only on the instantaneous acceleration would penalize this natural variation. Therefore, we low-pass filter the measured and the goal acceleration with the time constant τ and note them as $\bar{\alpha}_t$ and $\bar{g}_t$. We then define the acceleration reward and the velocity hold reward as

$$r_t^{\text{acc}} = \frac{\bar{\alpha}_t \left(2\bar{g}_t - \bar{\alpha}_t\right)}{\alpha_{\max}^2} \qquad \text{and} \qquad r_t^{\text{hold}} = \exp\left(-\frac{\|\hat{v}_t - v_t^{\text{r}}\|^2}{\sigma \, \|v^{\text{c}}\|^2}\right), \tag{3}$$

where σ is the temperature. The acceleration reward is the negative squared tracking error $-(\bar{\alpha}_t - \bar{g}_t)^2$, shifted such that a robot that stands still receives exactly zero and has to accelerate to get any reward. However, at a constant walking speed, the acceleration is zero, which is why the hold reward additionally tracks the velocity reference with the velocity estimate. The full reward $r_t = \Delta t \left(c_{\text{acc}} \, r_t^{\text{acc}} + c_{\text{hold}} \, r_t^{\text{hold}} + r_t^{\text{reg}}\right) - c_{\text{fall}} \, \mathbb{1}_t^{\text{fall}}$ adds a penalty for falling and the regularization terms r_t^{reg}, which keep the yaw rate low, the trunk upright and the upper body close to its nominal pose and penalize fast changes in the actions. The velocity estimate itself only enters the goal acceleration and the hold reward and is not part of the observation.

3.2 Sample-Efficient Learning

The position targets of the policy are tracked by Proportional-Derivative (PD) controllers. To make the joints follow the targets closely, we use a high gain setting with a proportional gain of 60 and a derivative gain of 3. With stiff and well-damped joints, the effect of every action becomes more predictable, which makes it easier to stabilize the robot early in training. We use XQC [17] as our RL algorithm, which is an off-policy actor-critic algorithm that learns sample-efficiently through a well-conditioned critic optimization with batch normalization [16], weight normalization and a distributional cross-entropy loss.

Naturally, a humanoid is left-right symmetric, and so is the locomotion task that we train on, so every collected transition has a mirrored counterpart that is equally valid [20], i.e. taking a step to the left under a certain observation is equivalent to taking a step to the right under the mirrored observation. We define a mirror operator M that swaps the left and the right leg and negates all lateral observations and actions, and we replace every transition (o_t, a_t, r_t, o_{t+1}) sampled from the replay buffer by $(Mo_t, Ma_t, r_t, Mo_{t+1})$ with a probability of 0.5. Thereby, the policy and the critic learn about both sides of the gait from every collected transition with no additional cost.

4 Experiments

We evaluate STRIDE on the Unitree G1 and Booster T1 humanoids in the MuJoCo simulator [24] and implement in RL-X [25]. The policy controls the 12 leg joints at 50 Hz, and the robot is commanded to walk forward at 0.5 m/s for 12 s per episode. To stay close to learning on a real robot, every run collects 250k transitions with a single robot, which corresponds to 83 minutes worth of samples. We report the time to walk and the falls during training, as they determine the cost of learning on a real robot. A run learns to walk once its mean episode length reaches 500 of the 600

Table 1: Runs that learn to walk within 83 minutes, their average time to walk, and the average number of falls over all seeds. Every setting is trained on at least two seeds.

	Unitree G1			Booster T1		
Method	Walk	Time [min]	Falls	Walk	Time [min]	Falls
STRIDE	2/2	13	443	2/2	13	437
w/o symmetry augmentation	2/2	21	646	2/2	18	583
low gain setting	4/4	22	689	2/2	66	2567
low gain setting, w/o symmetry	4/7	49	1942	1/3	82	3250
low gain setting, w/o symmetry, UTD ratio 4	2/2	30	855	2/2	53	2108
w/o symmetry, with CPG	2/2	20	628	2/2	18	579
shared policy	2/2	60	1760	0/2	–	3792

steps and its mean forward velocity reaches 0.4 m/s in three consecutive windows of 1,000 steps. The velocity of the simulator is only used for this evaluation. We use $\alpha_{\max} = 0.75\,\mathrm{m/s^2}$, $k_{\mathrm{P}} = 4$, $k_{\mathrm{I}} = 1$, $\tau = 0.25\,\mathrm{s}$, $\sigma = 0.25$, $c_{\mathrm{acc}} = 10$, $c_{\mathrm{hold}} = 5$, $c_{\mathrm{fall}} = 2$ and a UTD ratio of 2. For comparison, we also train without the symmetry augmentation and with a low gain setting, which uses PD gains of 45 and 1 for the G1 and of 25 and 0.6 for the T1, which are the default in RL-X.

Figure 2 shows that STRIDE reaches the commanded velocity on both humanoids within the first minutes of training. Every run learns to walk after 12 to 14 minutes worth of samples, which is earlier than any run of its ablations, and the final policies walk at 0.48 m/s and complete every episode. Learning with the low gain setting and without symmetry augmentation is far less reliable, as only half of these runs learn to walk within the budget and the ones that do need 56 minutes on average (see Table 1). Without the symmetry augmentation, the time to walk increases from 13 to 21 minutes on the G1 and to 18 minutes on the T1. With the low gain setting, the G1 needs 22 minutes and the T1 even 66 minutes. The number of falls stops increasing as soon as a policy walks, and across all runs that learn to walk, 99% of the falls happen before that point. Therefore, learning faster also means falling less, and STRIDE reduces the average number of falls during training from over 2000 to 440.

Finally, a higher UTD ratio trades computational efficiency for sample efficiency. Without the two components of STRIDE, increasing it from 2 to 4 lets every run learn to walk and reduces the time to walk from 49 to 30 minutes on the G1 and from 82 to 53 minutes on the T1. A CPG that generates alternating foot trajectories, on top of which the policy predicts residual joint targets [21, 11], does not change the time to walk with the high gain setting, so STRIDE does not seem to benefit from a gait prior. Furthermore, we train a single policy on both humanoids at once, which share the same observation and action space. With the shared policy, every G1 run learns to walk, while the T1 does not yet learn to walk within the training budget. In the current setup, we do not see positive transfer for training on both robots and sharing the collected transitions. As the shared policy has to infer which robot it controls, embodiment-aware architectures [6, 7] are a natural next step to explore.

5 Conclusion and Limitations

We introduced STRIDE, which learns humanoid locomotion from minutes of samples with a reward that only requires the onboard IMU. High joint gains and symmetry augmentation turn a learning problem that only half of the runs solve within the budget into one that every run solves in a fraction of that time. Our experiments are limited to simulation, where the acceleration of the IMU is free of noise and bias. On a real robot, the IMU also measures gravity and its errors accumulate, so we expect that the hold reward will accumulate these errors as well. In future work, we will bring STRIDE to the physical G1 and T1, add omnidirectional commands and combine it with embodiment-aware network architectures to improve the transfer when training on both robots together.

Acknowledgments

We acknowledge EuroHPC Joint Undertaking for awarding us access to LUMI at CSC, Finland, MeluXina at LuxProvide, Luxembourg, Vega at IZUM, Slovenia, and Arrhenius at NAISS, Sweden, through the Benchmark Access project EHPC-BEN-2026B08-075 for LUMI and the Development Access project EHPC-DEV-2026D08-135 for the other three systems. LUMI is hosted by CSC (Finland) and the LUMI consortium. The authors gratefully acknowledge the HPC RIVR consortium and EuroHPC JU for funding this research by providing computing resources of the HPC system Vega at the Institute of Information Science. Some of the experiments were performed on the Luxembourg national supercomputer MeluXina, and the authors gratefully acknowledge the LuxProvide teams for their expert support. Computational resources were provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), funded by the Swedish Research Council. The authors gratefully acknowledge the computing time granted by the KISSKI project. The calculations for this research were conducted with computing resources under the project kisski-merl. The authors gratefully acknowledge the computing time provided to them on the high-performance computer Lichtenberg II at the NHR Center NHR4CES at TU Darmstadt (project number p0028682). This is funded by the Federal Ministry of Research, Technology and Space, and the state governments participating on the basis of the resolutions of the GWK for national high performance computing at universities. Some of the experiments ran on the hessian.AI cluster 43, and we gratefully acknowledge support from the hessian.AI Service Center (funded by the Federal Ministry of Research, Technology and Space, BMFTR, grant no. 16IS22091) and the hessian.AI Innovation Lab (funded by the Hessian Ministry for Digital Strategy and Innovation, grant no. S-DIW04/0013/003).

References

[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), 2019.

[2] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath. Real-world humanoid locomotion with reinforcement learning. *Science Robotics*, 2024.

[3] M. Stasica, A. Bick, N. Bohlinger, O. Mohseni, M. J. A. Fritzsche, C. Hübler, J. Peters, and A. Seyfarth. Bridge the gap: Enhancing quadruped locomotion with vertical ground perturbations. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. doi:10.1109/IROS60139.2025.11247549.

[4] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 91–100, 2022.

[5] N. Bohlinger and J. Peters. Massively scaling explicit policy-conditioned value functions. *arXiv preprint arXiv:2502.11949*, 2025.

[6] N. Bohlinger, G. Czechmanowski, M. P. Krupka, P. Kicki, K. Walas, J. Peters, and D. Tateo. One policy to run them all: an end-to-end learning approach to multi-embodiment locomotion. In *Conference on Robot Learning (CoRL)*, volume 270 of *Proceedings of Machine Learning Research*, pages 3356–3378, 2024.

[7] N. Bohlinger and J. Peters. Multi-embodiment locomotion at scale with extreme embodiment randomization. *arXiv preprint arXiv:2509.02815*, 2025.

[8] N. Bohlinger and J. Peters. Active embodiment identification with reinforcement learning for legged robots. *arXiv preprint arXiv:2605.08020*, 2026.

[9] D. Li, B. Ai, N. Bohlinger, J. Peters, H. Su, and H. I. Christensen. Rapid embodiment adaptation for quadrupedal locomotion. *arXiv preprint arXiv:2608.01506*, 2026.

[10] L. Smith, I. Kostrikov, and S. Levine. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. In *Robotics: Science and Systems (RSS)*, 2023.

[11] N. Bohlinger, J. Kinzel, D. Palenicek, Ł. Antczak, and J. Peters. Gait in eight: Efficient on-robot learning for omnidirectional quadruped locomotion. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8790–8797, 2025. doi: 10.1109/IROS60139.2025.11246447.

[12] D. Aditya, J. Huang, N. Bohlinger, P. Kicki, K. Walas, J. Peters, M. Luperto, and D. Tateo. Robust localization, mapping, and navigation for quadruped robots. In *European Conference on Mobile Robots (ECMR)*, pages 1–8, 2025. doi:10.1109/ECMR65884.2025.11163249.

[13] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine. Learning to walk via deep reinforcement learning. In *Robotics: Science and Systems (RSS)*, 2019.

[14] P. Wu, A. Escontrela, D. Hafner, K. Goldberg, and P. Abbeel. DayDreamer: World models for physical robot learning. In *Conference on Robot Learning (CoRL)*, 2022.

[15] C. Li, A. Krause, and M. Hutter. Robotic world model: A neural network simulator for robust policy optimization in robotics. *arXiv preprint arXiv:2501.10100*, 2025.

[16] A. Bhatt, D. Palenicek, B. Belousov, M. Argus, A. Amiranashvili, T. Brox, and J. Peters. CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *International Conference on Learning Representations (ICLR)*, 2024.

[17] D. Palenicek, F. Vogt, J. Watson, I. Posner, and J. Peters. XQC: Well-conditioned optimization accelerates deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026.

[18] P. D'Oro, M. Schwarzer, E. Nikishin, P.-L. Bacon, M. G. Bellemare, and A. Courville. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *International Conference on Learning Representations (ICLR)*, 2023.

[19] L. Smith, Y. Cao, and S. Levine. Grow your limits: Continuous improvement with real-world RL for robotic locomotion. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 10829–10836, 2024.

[20] M. Mittal, N. Rudin, V. Klemm, A. Allshire, and M. Hutter. Symmetry considerations for learning task symmetric robot policies. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 7433–7439, 2024.

[21] G. Bellegarda and A. Ijspeert. CPG-RL: Learning central pattern generators for quadruped locomotion. *IEEE Robotics and Automation Letters*, 2022.

[22] B. Ai, L. Dai, N. Bohlinger, D. Li, T. Mu, Z. Wu, K. Fay, H. I. Christensen, J. Peters, and H. Su. Towards embodiment scaling laws in robot locomotion. In *Conference on Robot Learning (CoRL)*, volume 305 of *Proceedings of Machine Learning Research*, pages 3483–3515, 2025.

[23] N. Bohlinger and J. Peters. Shape your body: Value gradients for multi-embodiment robot design. In *Conference on Robot Learning (CoRL)*, 2026.

[24] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[25] N. Bohlinger and K. Dorer. RL-X: A deep reinforcement learning library (not only) for RoboCup. In *RoboCup 2023: Robot World Cup XXVI*, volume 14140 of *Lecture Notes in Computer Science*, pages 228–239. Springer, 2024.